Enterprise Azure Hybrid Networking and IaC Lab Guide

A refresher guide for practicing hub-and-spoke networking, Bastion, internal load balancing, Azure Route Server with BGP, ExpressRoute design, Terraform CI/CD, state control, module registry patterns, Git integration, monitoring, cost governance, and minimum viable change management.

Designed for a Mac with Apple Silicon: use the Mac as the admin workstation and use Azure to host the simulated on-prem and target cloud environments.

Lab area	What you practice	Why it matters
Mac tooling	Azure CLI, PowerShell, Terraform, Git, VS Code	Builds the daily engineer workflow without fighting Apple Silicon VM limitations.
Terraform state	Remote state in Azure Storage	Allows safe collaboration, state locking, and controlled infrastructure changes.
Hub-and-spoke	Hub VNet, spokes, peering, subnets	Matches common enterprise Azure network segmentation patterns.
Bastion	Private VM access without VM public IPs	Reduces public RDP/SSH exposure.
Load balancer	Internal load-balanced app	Teaches health probes, backend pools, and high availability.
Route Server + BGP	Dynamic routes with Linux NVA	Builds routing troubleshooting skill beyond basic VNets.
ExpressRoute design	Private connectivity design exercise	Prepares you to discuss hybrid connectivity without needing a real circuit.
CI/CD + modules	Plan/apply pipeline, module versioning	Builds repeatable platform engineering habits.
Governance	Tags, budgets, change register	Connects technical changes to cost, accountability, and operations.

How to use this guide

Treat each phase as a mini project. For every phase, create a change request, run Terraform plan, implement the change, validate it, capture evidence, and update the change register. This mirrors how a senior cloud engineer would work in a real organization.

- Do not deploy every component on day one. Some services are billable and can add cost quickly.
- Start with networking and state control before advanced routing.
- Use simulated on-prem in Azure instead of trying to host x64 Windows Server workloads locally on Apple Silicon.
- Keep screenshots and command outputs as evidence for interviews and portfolio notes.

Phase 0 - Prepare the Mac admin workstation

What you build: Install Azure CLI, PowerShell 7, Az PowerShell module, Terraform, Git, VS Code, jq, and Microsoft Remote Desktop if needed.

Why this matters: Your Mac is the operator workstation. The lab infrastructure runs in Azure, but all engineering work should start from a repeatable local toolchain.

Steps

- Install tools with Homebrew.
- Log in with az login.
- Select the correct subscription.
- Run scripts/preflight_check.sh.

Validation: az account show --output table; terraform version; git --version.

Common mistakes: Do not try to run a complete x64 Windows Server estate locally on Apple Silicon. Use Azure VMs for Windows Server labs.

Phase 1 - Bootstrap Terraform remote state

What you build: Create a resource group, storage account, and blob container for Terraform state.

Why this matters: Remote state lets teams collaborate, prevents accidental local state loss, and supports state locking. Terraform state can contain sensitive values, so it needs controlled storage and access.

Steps

- Run scripts/bootstrap_tfstate.sh.
- Copy backend.tf.example to backend.tf.
- Fill in storage account, container, and key.
- Run terraform init.

Validation: Confirm the storage account and tfstate container exist. Confirm terraform init succeeds.

Common mistakes: Do not commit local .tfstate files. Do not share state storage keys broadly.

Phase 2 - Create the Git workflow

What you build: Create a repo, commit the Terraform skeleton, use feature branches, and document pull request workflow.

Why this matters: Git gives traceability. Every infrastructure change should have a reason, reviewer, and rollback path.

Steps

- git init
- Create main branch.
- Create feature branch.
- Commit baseline.
- Open pull request in GitHub or Azure DevOps.

Validation: Repo has environments, modules, pipelines, docs, and templates. No secrets or state files are committed.

Common mistakes: Do not apply from random untracked local files. Keep plans tied to code versions.

Phase 3 - Deploy management resources

What you build: Deploy management resource groups, Log Analytics workspace, recovery vault placeholder, and required tags.

Why this matters: Enterprise platforms need operational services before workloads. Monitoring, backup, tagging, and state should not be afterthoughts.

Steps

- Create change request CHG-2026-001.
- terraform plan.
- Review resources and tags.
- terraform apply.

Validation: Resource groups exist and required tags are present.

Common mistakes: Do not deploy workloads before management and governance basics exist.

Phase 4 - Deploy hub-and-spoke VNets

What you build: Create hub VNet, app spoke, data spoke, shared services spoke, and simulated on-prem VNet.

Why this matters: Hub-and-spoke is a common Azure network pattern for centralizing connectivity, security inspection, shared services, and governance.

Steps

- Deploy hub VNet with AzureBastionSubnet, RouteServerSubnet, GatewaySubnet, management subnet, and NVA subnet.
- Deploy spokes.
- Create peering.
- Document address spaces.

Validation: az network vnet list; peerings show Connected; address spaces do not overlap.

Common mistakes: Avoid overlapping CIDRs. Plan IP space before deploying.

Phase 5 - Add Azure Bastion for secure access

What you build: Deploy Bastion into AzureBastionSubnet and connect to private VMs without public IPs.

Why this matters: Bastion provides RDP/SSH access without exposing management ports directly to the internet, which is a basic security improvement for VM administration.

Steps

- Uncomment Bastion module.
- Run Terraform plan/apply.
- Create test VM without public IP.
- Connect through Azure Portal Bastion.

Validation: VM has no public IP. Bastion connection works. Direct public RDP/SSH is not available.

Common mistakes: Do not place Bastion in a random subnet. It must use AzureBastionSubnet with the required sizing.

Phase 6 - Add internal load balancer and web VMs

What you build: Create two web VMs in the app spoke and place them behind an internal Standard Load Balancer.

Why this matters: This teaches backend pools, frontend IPs, health probes, and app availability without exposing the app publicly.

Steps

- Deploy two web VMs.
- Install IIS or Nginx.
- Deploy internal load balancer.
- Add backend NICs.
- Create probe and rule.

Validation: Connect from another private VM to the load balancer IP. Stop one web service and confirm failover behavior.

Common mistakes: Health probes must match the service port. NSGs must allow probe and app traffic.

Phase 7 - Add Azure Route Server and BGP NVA

What you build: Deploy Azure Route Server and a Linux NVA running FRRouting. Configure BGP peering.

Why this matters: Route Server and BGP teach dynamic routing. This is the difference between basic Azure networking and enterprise routing troubleshooting.

Steps

- Deploy Route Server in RouteServerSubnet.
- Deploy Linux NVA in hub NVA subnet.
- Install/configure FRR.
- Peer NVA with both Route Server instances.
- Advertise simulated on-prem prefix.

Validation: Route Server peering is established. Effective routes on spoke NICs show learned routes. Connectivity follows expected path.

Common mistakes: Do not advertise broad or wrong prefixes. Route leaks can break connectivity. Keep the lab isolated.

Phase 8 - Practice ExpressRoute design without a real circuit

What you build: Create an ExpressRoute design diagram and compare it to VPN/peering simulation.

Why this matters: Real ExpressRoute usually requires a provider. In interviews, you still need to explain when and why it is used: private connectivity, predictable routing, bandwidth, and enterprise hybrid integration.

Steps

- Read expressroute_design_practice.md.
- Draw on-prem router, provider, ExpressRoute circuit, ExpressRoute gateway, hub VNet, spokes.
- Define fallback path and route filtering approach.

Validation: You can explain private peering, gateway role, provider dependency, routing, failover, and why ExpressRoute differs from public internet VPN.

Common mistakes: Do not claim ExpressRoute is just a bigger VPN. It is private connectivity through a provider.

Phase 9 - Add CI/CD for Terraform

What you build: Use GitHub Actions or Azure DevOps to run terraform fmt, init, validate, and plan.

Why this matters: CI/CD makes infrastructure changes reviewable and repeatable. Plan output becomes part of the change evidence.

Steps

- Add pipeline YAML.
- Configure Azure auth.
- Run on pull request.
- Require plan review before apply.

Validation: Pipeline succeeds for fmt, validate, init, and plan.

Common mistakes: Do not store long-lived secrets if you can use workload identity federation/OIDC.

Phase 10 - Create Terraform module registry practice

What you build: Start with local modules, then split modules into separate Git repos with semantic version tags.

Why this matters: A module registry pattern lets teams reuse known-good infrastructure patterns instead of copying random code.

Steps

- Document module inputs/outputs.
- Tag a module v0.1.0.
- Reference a Git tag from an environment.
- Make a backward-compatible update v0.2.0.

Validation: Environment can reference a specific module version. Module README explains usage.

Common mistakes: Do not make breaking variable changes without a major version bump.

Phase 11 - Add monitoring, cost governance, and change control

What you build: Apply tags, query inventory, create budgets, document changes, and produce a simple showback report.

Why this matters: Senior engineers connect infrastructure to operations: who owns it, what it costs, how it is monitored, and what changed.

Steps

- Apply tagging standard.
- Run Resource Graph queries.
- Create budget alerts.
- Update change register.
- Capture validation evidence.

Validation: Resources have required tags. Budget exists. Change register is updated. Cost report groups by tag once cost appears.

Common mistakes: Cost Management reports only billable usage. Free/near-free lab resources may not show much cost.

Core commands

Use these commands as quick refreshers while practicing. Replace placeholder values before running.

```
# Login and subscription
az login
az account list --output table
az account set --subscription "<subscription-id>"

# Bootstrap remote state
cd enterprise_azure_hybrid_networking_iac_lab
bash scripts/bootstrap_tfstate.sh

# Terraform deploy flow
cd terraform/environments/dev
cp backend.tf.example backend.tf
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan

# Validate VNets
az network vnet list --query "[].{Name:name,RG:resourceGroup,Address:addressSpace.addressPrefixes}" --output table

# Validate effective routes
az network nic show-effective-route-table -g <resource-group> -n <nic-name> --output table
```

Minimum viable change management

Do not spend weeks building a full ITIL process for a lab. Use a simple change register and a one-page change request. The minimum requirement is traceability: what changed, why it changed, who approved it, how it was validated, and how it can be rolled back.

Change type	Use for	Approval
Standard	Repeatable low-risk changes such as adding approved tags or running validation queries	Cloud lead
Normal	Networking, routing, Bastion, policy, or production-like changes	Cloud/network owner
Emergency	Urgent fix after a failed deployment or broken route/policy	After-action review

Interview story you are building

After finishing the lab, you should be able to say:

"I built an enterprise-style Azure lab using Terraform, remote state, Git-based version control, and CI/CD validation. The network followed a hub-and-spoke model with Bastion for private VM access, internal load balancing for app availability, and Azure Route Server with a Linux NVA over BGP to practice dynamic routing. I simulated hybrid connectivity and ExpressRoute design with a separate on-prem VNet. I also added tagging, cost governance, monitoring, and a lightweight change-management process."

Reference notes

The lab is aligned to Microsoft guidance for the services below. Use the linked official documentation for exact service limits, SKU requirements, and current pricing before deploying in a real environment.

Topic	Why it is referenced
Azure Route Server	BGP route reflector behavior and NVA peering pattern.
Route injection into spokes	Understanding how learned routes propagate in hub-and-spoke designs.
Azure Bastion	Secure RDP/SSH without public IPs on VMs.
ExpressRoute	Private connectivity from on-premises networks to Microsoft cloud through a provider.
Terraform remote state in Azure Storage	Team-safe state storage, state locking, and secure backend practice.
Azure hub-spoke architecture	Reference pattern for centralized shared services and spoke isolation.
Azure Load Balancer	Internal load balancing and health-probe based availability.